Appendix C: AI-Assisted Secure Coding

Objective

This appendix lists organizational controls for using AI coding tools safely. The range is baseline to advanced. Scope is coding, code review, and the rest of the SSDLC.

AC.1 AI-Assisted Secure-Coding Workflow

AI tooling has to slot into the existing SSDLC without weakening any of the security gates already in place. Equally important: write down the adversarial-AI threat scenarios that justify each guardrail. Doing this up front is much easier than reconstructing it after the fact.

#	Description	Level
AC.1.1	**Verify that** a written workflow says when AI tools may generate, refactor, or review code. The workflow names the approved tools, the prohibited use cases, and the data classifications that are allowed as input.	1
AC.1.2	**Verify that** the workflow covers every SSDLC phase from design and implementation through code review, testing, deployment, and post-deployment monitoring, and names the security gates that stay mandatory whether AI was involved or not.	2
AC.1.3	**Verify that** the workflow names the adversarial-AI threat scenarios it is built to mitigate. The list should cover prompt injection delivered through PR content, AI-generated supply-chain payloads, autonomous agents approving their own work, fork-PR secret exfiltration, and compromise of the model supply chain.	2
AC.1.4	**Verify that** metrics are collected on AI-produced and AI-mediated code, and that the results are compared against a human-only baseline. Vulnerability density, mean-time-to-detect, AI-attributable defect rate, prompt-injection detection rate, and fork-PR rejection rate are all useful.	3

Mappings & References:

- **AC.1.1:** NIST SSDF PO.1 (Define Security Requirements for Software Development); ISO/IEC 42001 Clauses 6, 8; OWASP SAMM Strategy & Metrics (SM), Policy & Compliance (PC).
- **AC.1.2:** NIST SSDF PW.1, PW.7; OWASP SAMM Education & Guidance (EG); ISO/IEC 5338 Clause 6.
- **AC.1.3:** MITRE ATLAS (Reconnaissance & Initial Access tactics); NIST AI 600-1 GOVERN; OWASP LLM Top 10 (2025) LLM03; OWASP Agentic Top 10 (2026) ASI04.
- **AC.1.4:** NIST AI RMF MEASURE; ISO/IEC 42001 Clause 9; OWASP SAMM Strategy & Metrics (SM).

AC.2 AI Tool Qualification & Threat Modeling

Do not adopt an AI coding tool until it has been evaluated. Three areas in particular: its security capabilities, its resistance to adversarial input, and the risk inherited from its supply chain.

#	Description	Level
AC.2.1	**Verify that** every AI tool, whether it is an assistant, a reviewer, an agent, or an MCP server, has a threat model. The threat model covers misuse, model inversion, training-data leakage, prompt injection from untrusted input, insecure output handling, excessive agency, and risk inherited from its dependency chain.	1
AC.2.2	**Verify that** the evaluation of each tool covers the local components (static and dynamic analysis), the SaaS endpoints (TLS, AuthN/AuthZ, logging, data residency), and the vendor's model supply chain (training-data provenance, fine-tune history, RAG sources). Each of these is reviewed and the review is written down.	2
AC.2.3	**Verify that** each tool goes through adversarial robustness testing before onboarding. The testing is repeated after any material change to the model or to the system prompts. Coverage includes automated prompt-injection probes, jailbreak suites, and indirect-injection corpora delivered through realistic PR and issue surfaces.	2
AC.2.4	**Verify that** evaluations follow a recognized framework such as NIST AI RMF, NIST AI 600-1 Generative AI Profile, or ISO/IEC 42001. Evaluations are repeated after a major version change, a vendor incident, or new threat intelligence relevant to the tool class.	3

Mappings & References:

- **AC.2.1:** OWASP LLM Top 10 (2025) LLM01, LLM06; OWASP Agentic Top 10 (2026) ASI01, ASI02, ASI03; AISVS C9; MITRE ATLAS (Threat modeling).
- **AC.2.2:** OWASP LLM Top 10 (2025) LLM03; OWASP Agentic Top 10 (2026) ASI04; NIST SSDF PO.1, PO.5; ISO/IEC 42001 Clause 8.
- **AC.2.3:** MITRE ATLAS (Adversarial ML testing); AISVS C2.1, C11.1; NIST AI 600-1 MEASURE.
- **AC.2.4:** ISO/IEC 42001 Clause 9.2; NIST AI RMF GOVERN.

AC.3 Secure Prompt & Context Management

Two goals in this family. First: stop secrets, proprietary code, and personal data from leaking into prompts. Second: treat any content sourced from the repository, a PR, or a third party as untrusted input. Any of it can carry a prompt-injection payload, and most of it usually does not, which is part of what makes the rare hostile case easy to miss.

> **Relationship to AISVS C2.1:** AC.3.3, AC.3.4, and AC.3.5 apply AISVS C2.1 (Prompt Injection Defenses) to the secure-coding case. If a finding here is something that C2.1 verification did not already close, count it as an additional gap (specific to coding-tool prompt construction). If C2.1 already closed it, do not count it twice.

#	Description	Level
AC.3.1	**Verify that** written guidance forbids putting secrets, credentials, PII, or classified data in any prompt sent to an AI tool. The guidance is enforced in pre-commit hooks, IDE integrations, and CI.	1
AC.3.2	**Verify that** technical controls automatically strip sensitive material from any context window sent to an AI tool. Client-side redaction, approved context filters, and secret scanners with pre-prompt hooks all qualify.	1
AC.3.3	**Verify that** any externally sourced context being fed to an AI tool is treated as untrusted and screened for prompt injection before it reaches the prompt. Sources to cover: PR descriptions and comments, fork-supplied diffs, issue bodies, commit messages, third-party documentation, web search results, and MCP tool outputs.	1
AC.3.4	**Verify that** the AI tool enforces an instruction hierarchy, with system and developer messages taking precedence over untrusted repository content. This hierarchy has to hold across multi-turn conversations and tool-augmented workflows.	1
AC.3.5	**Verify that** input length controls stop untrusted PR or repository content from crowding system instructions or safety directives out of the effective context window. Oversized inputs are rejected outright. Silent truncation is not acceptable.	2
AC.3.6	**Verify that** prompts and AI responses are encrypted in transit and at rest, and retained per the data-classification policy. Tenants and projects are cryptographically separated from each other.	3

Mappings & References:

- **AC.3.1:** OWASP LLM Top 10 (2025) LLM02 (Sensitive Information Disclosure); OWASP ASVS v5 V14 (Data Protection); ISO/IEC 27001:2022 A.8.12 (Data Leakage Prevention).
- **AC.3.2:** AISVS C2.2; OWASP LLM Top 10 (2025) LLM02; NIST SSDF PW.3.
- **AC.3.3:** AISVS C2.1; OWASP LLM Top 10 (2025) LLM01; OWASP Agentic Top 10 (2026) ASI06; MITRE ATLAS (Indirect prompt injection).
- **AC.3.4:** AISVS C2.1.2; OWASP LLM Top 10 (2025) LLM01; CISA Secure by Design.
- **AC.3.5:** OWASP LLM Top 10 (2025) LLM10; AISVS C2.1.4.
- **AC.3.6:** OWASP ASVS v5 V6 (Cryptography), V14 (Data Protection); ISO/IEC 27001:2022 A.8.24 (Use of Cryptography).

AC.4 Validation of AI-Generated Code

Catch the vulnerabilities AI output introduces. Fix them before the code reaches a merge or a deployment.

#	Description	Level
AC.4.1	**Verify that** AI-generated code always goes through code review by a qualified human engineer. The reviewer must not be the same identity that asked for the AI generation in the first place (separation of duties). And the AI agent itself does not count as the human reviewer.	1
AC.4.2	**Verify that** automated security testing runs on every pull request containing AI-generated code: SAST, IAST, DAST, secret scanning, IaC scanning, and SCA. Where the scanner supports them, AI-attribution-aware rules are turned on.	2
AC.4.3	**Verify that** pull requests containing AI-generated code are blocked from merging when an automated scan surfaces a critical security finding, defined as CVSS >= 9.0 or the equivalent threshold in the organization's vulnerability severity policy. Bypassing the block requires a written exception approved by an authorized human.	2
AC.4.4	**Verify that** security-critical files require an elevated review threshold when AI generated or modified them: two-person review, security-team sign-off, or stricter. Security-critical files here include authentication, authorization, and cryptography code; IAM policy; CI/CD workflow definitions; deployment manifests; and sandbox or network policy artifacts.	2
AC.4.5	**Verify that** differential fuzz testing or property-based tests cover the security-critical behaviors of AI-generated code: input validation, authorization logic, and deserialization safety.	3

Mappings & References:

- **AC.4.1:** NIST SSDF PW.7; OWASP ASVS v5 V10 (Coding Quality); ISO/IEC 27001:2022 A.5.3 (Segregation of Duties).
- **AC.4.2:** NIST SP 800-204D (Pipeline scanning controls); SLSA v1.2 Build Track L2; OWASP SAMM Security Testing (ST).
- **AC.4.3:** OWASP CI/CD Top 10 CICD-SEC-04 (Poisoned Pipeline Execution); NIST SSDF PW.7, PW.8.
- **AC.4.4:** NIST SSDF PW.4, PW.7; OWASP CI/CD Top 10 CICD-SEC-01 (Insufficient Flow Control); ISO/IEC 27001:2022 A.8.32 (Change Management).
- **AC.4.5:** NIST SSDF PW.8; OWASP ASVS v5 V11 (Business Logic).

AC.5 Explainability & Traceability of Code Suggestions

Auditors, defenders, and the developers themselves need to be able to see why a given AI suggestion was made, and how it ended up in a shipped artifact.

#	Description	Level
AC.5.1	**Verify that** prompt-and-response pairs are logged with stable correlation identifiers, so that an investigator can later replay the whole chain: prompt → response → commit → build → deployment.	1
AC.5.2	**Verify that** developers can pull up the citations (training snippets, retrieved documents, MCP tool outputs) that support a suggestion, and that the citation chain travels with the artifact.	3
AC.5.3	**Verify that** explainability reports, AI-event logs, and citation records are kept in tamper-evident storage (append-only, WORM, or an immutable log store) and are referenced during security reviews.	3

Mappings & References:

- **AC.5.1:** ISO/IEC 42001 Clause 7.5 (Documented Information); OWASP ASVS v5 V8 (Logging); NIST SP 800-218A (Generative AI logging guidance).
- **AC.5.2:** NIST AI RMF MEASURE; OWASP LLM Top 10 (2025) LLM03.
- **AC.5.3:** ISO/IEC 27001:2022 A.8.15 (Logging); NIST AI 600-1 MEASURE; ISO/IEC 42001 (traceability).

AC.6 Continuous Feedback, Adversarial Testing & Model Fine-Tuning

Improve model security over time. Watch for negative drift. Keep red-teaming the AI tooling. The red-team scope in this family is the AI tooling itself; the underlying systems and services the tooling depends on are handled by separate programs.

#	Description	Level
AC.6.1	**Verify that** developers and reviewers can flag insecure or non-compliant suggestions, and that each flag is tracked to closure with links back to the originating prompt and response and forward to any downstream artifacts.	1
AC.6.2	**Verify that** aggregated feedback feeds into periodic system-prompt updates or retrieval-augmented generation against vetted secure-coding corpora (OWASP Cheat Sheets, internal coding standards). Where the organization controls model training infrastructure, fine-tuning on the same feedback corpus is also required.	2
AC.6.3	**Verify that** scheduled red-team exercises target the AI tooling itself. The exercises include direct and indirect prompt-injection probes delivered through realistic PR, issue, and comment surfaces, jailbreak corpora, and supply-chain payload generation. Findings are remediated under tracked severity SLAs.	2
AC.6.4	**Verify that** a closed-loop evaluation harness runs regression tests after every fine-tune, system-prompt change, or model upgrade. Security metrics must meet or exceed the prior baseline before deployment.	3

Mappings & References:

- **AC.6.1:** NIST AI RMF MANAGE; ISO/IEC 42001 Clause 10; OWASP SAMM Defect Management (DM).
- **AC.6.2:** OWASP LLM Top 10 (2025) LLM03; NIST SSDF PO.3.
- **AC.6.3:** MITRE ATLAS (Adversarial ML lifecycle); NIST AI 600-1 MEASURE 2.7; OWASP SAMM Security Testing (ST).
- **AC.6.4:** ISO/IEC 42001 Clause 9.1; NIST AI RMF MEASURE.

AC.7 AI-Generated Infrastructure & Pipeline Artifacts

Infrastructure code, CI/CD workflow files, deployment manifests, and security policy artifacts each have outsized impact when they are wrong. When AI has generated them, the validation needs to be correspondingly stricter than for ordinary application code.

#	Description	Level
AC.7.1	**Verify that** AI-generated or AI-modified artifacts are clearly labeled and tracked as such. Artifact classes in scope include infrastructure-as-code (Terraform, CloudFormation, Pulumi, Bicep), CI/CD workflow files (GitHub Actions, GitLab CI, Jenkinsfile, Argo Workflows, Tekton), container and orchestration manifests (Dockerfile, Kubernetes, Helm), and security policy artifacts (IAM, OPA/Rego, NetworkPolicy, admission controllers).	1
AC.7.2	**Verify that** AI-generated infrastructure and pipeline configurations require human review and approval before they run in any environment beyond a hermetic sandbox.	2
AC.7.3	**Verify that** AI-generated infrastructure and workflow changes pass policy-as-code enforcement (OPA, Conftest, Checkov, tfsec, KICS, kube-linter) at the same level as, or stricter than, human-authored changes. Policy violations block promotion.	2
AC.7.4	**Verify that** changes to high-impact pipeline trigger configurations require both dual control and a security-team review, no matter who or what produced the change. The configurations in scope include GitHub Actions pull_request_target and workflow_run, self-hosted runner labels, workflow permissions: blocks, OIDC trust policies, and secret-environment mappings.	2
AC.7.5	**Verify that** drift detection compares deployed infrastructure and live workflow configurations against signed, AI-attributed baselines, and alerts on any unauthorized modification.	3

Mappings & References:

- **AC.7.1:** OWASP CI/CD Top 10 CICD-SEC-05 (Insufficient PBAC); SLSA v1.2 Build Track provenance; NIST SSDF PW.1.
- **AC.7.2:** NIST SP 800-204D (Approval gating); OWASP CI/CD Top 10 CICD-SEC-01; ISO/IEC 27001:2022 A.8.32 (Change Management).
- **AC.7.3:** OWASP ASVS v5 V10 (CI/CD Deployment Security); OWASP CI/CD Top 10 CICD-SEC-07 (Insecure System Configuration); NIST SSDF PW.4.
- **AC.7.4:** OWASP CI/CD Top 10 CICD-SEC-01, CICD-SEC-02; GitHub Security Lab "Preventing pwn requests" series; NIST SP 800-204D (Pipeline governance).
- **AC.7.5:** NIST SP 800-204D (Continuous monitoring); ISO/IEC 27001:2022 A.8.19.

AC.8 Autonomous Agent Change Control Constraints

Autonomous AI agents that generate code or configuration get the same separation-of-duties treatment that humans do. They cannot approve, merge, or promote their own work. This applies at the policy layer and at the technical layer.

#	Description	Level
AC.8.1	**Verify that** autonomous agents cannot approve, merge, sign, or deploy artifacts that they themselves generated, and that this constraint is enforced by the source-control system, the CI system, and the artifact registry. Policy alone does not satisfy this control.	1
AC.8.2	**Verify that** AI systems run with scoped, non-human identities (service accounts, workload identities, OIDC-issued ephemeral tokens), and that those identities cannot be used to promote their own generated artifacts across environments.	2
AC.8.3	**Verify that** autonomous agents cannot bypass branch protection, required reviews, required status checks, signed-commit requirements, or merge queues. Any attempt by an agent to change these settings raises a security alert.	2
AC.8.4	**Verify that** separation of duties holds across the stages of an AI-generated change. Each stage (generation, review, approval, deployment) is performed by a distinct principal, whether human or system.	3

Mappings & References:

- **AC.8.1:** OWASP Agentic Top 10 (2026) ASI03 (Identity and Privilege Abuse), ASI10 (Rogue Agents); OWASP ASVS v5 V10; NIST SP 800-53r5 AC-5 (Separation of Duties).
- **AC.8.2:** NIST SP 800-207 (Zero Trust Architecture); OWASP CI/CD Top 10 CICD-SEC-02; ISO/IEC 27001:2022 A.5.15 (Access Control).
- **AC.8.3:** OWASP CI/CD Top 10 CICD-SEC-01; GitHub Docs (Branch protection rules and rulesets); OWASP Agentic Top 10 (2026) ASI03.
- **AC.8.4:** NIST SSDF PO.2; ISO/IEC 27001:2022 A.5.3; NIST SP 800-53r5 AC-5.

AC.9 AI Artifact Origin Validation for Deployment

Deployment and promotion pipelines need to validate the cryptographic origin and the generation history of AI-generated artifacts. They do this before letting the artifact through.

#	Description	Level
AC.9.1	**Verify that** AI-generated artifacts carry signed origin and generation metadata (in-toto or SLSA provenance attestations, AI BOM entries) identifying the AI system that produced them, the generation context, the humans involved, and the associated audit records.	2
AC.9.2	**Verify that** deployment pipelines check the presence, signature, and integrity of origin and generation metadata on AI-generated artifacts before promotion, using a trusted verifier (Sigstore/cosign, in-toto verification).	3
AC.9.3	**Verify that** artifacts are rejected at deployment and quarantined for review when they are missing required origin and generation information, signed by untrusted keys, or produced by an unapproved AI system or environment.	3

Mappings & References:

- **AC.9.1:** SLSA v1.2 (Provenance attestations); CycloneDX ML-BOM; in-toto Attestation Framework.
- **AC.9.2:** SLSA v1.2 (Verification Summary Attestations); Sigstore/cosign (Signature verification); OWASP SCVS.
- **AC.9.3:** SLSA v1.2 (Verifier requirements); NIST SP 800-204D (Promotion gating).

AC.10 Generation Audit Trail Completeness and Validation

AI-generated artifacts need complete and consistent origin and generation records, validated before integration or deployment. The reason matters. Policy-based enforcement of origin tracking only works if the recorded information is itself complete and consistent. When records are missing fields, or when the fields they do have contradict each other, detections get missed and enforcement opens gaps. So origin tracking is treated as a first-class requirement here, and validated before an artifact is accepted.

#	Description	Level
AC.10.1	**Verify that** AI-generated artifacts carry the required origin and generation fields: model identity and version, tool or agent identity, generation context, prompt hash, human involvement, session identifiers, and correlation IDs.	1
AC.10.2	**Verify that** origin and generation metadata is checked for completeness and consistency: no missing or ambiguous fields, values normalized to a single representation, and a signature chain that validates back to a trusted root.	2
AC.10.3	**Verify that** artifacts with incomplete, inconsistent, or unverifiable origin and generation metadata are rejected before merge or deployment, and that the rejection event is logged so trends can be tracked. Rejection happens on the verifier side, against the attestation or proof model defined in SLSA and the verification criteria in ISO/IEC 42001.	3

Mappings & References:

- **AC.10.1:** CycloneDX ML-BOM schema; NIST SP 800-218A (Generative AI provenance); ISO/IEC 42001 Clause 7.5.
- **AC.10.2:** OWASP SCVS (Provenance and Pedigree); SLSA v1.2 VSA verification.
- **AC.10.3:** SLSA v1.2 (Verifier-side enforcement); ISO/IEC 42001 Clause 9.

AC.11 AI Code-Review & Assistant Bot Hardening

AI code-review bots, PR-comment bots, MCP-driven assistants (Model Context Protocol), and IDE copilots are all reachable through untrusted repository content. The reachable surfaces include PR diffs, descriptions, comments, issues, and any workflow files supplied from a fork. This family covers the case where an attacker uses one of those surfaces to push a defender's own AI agent into approving, ignoring, or actively assisting a supply-chain attack.

> **Relationship to AISVS C2.1, C9.3, and C9.5:** AC.11.1 through AC.11.5 are applications of three AISVS chapter controls to the specific case of AI code-review and assistant bots operating over untrusted PR content. The three chapter controls are C2.1 (Prompt Injection Defenses), C9.3 (Component Isolation and Tool Authorization), and C9.5 (Agent Authorization, Delegation, and Continuous Enforcement). The appendix restates each one with bot-specific guidance. Counting rule is the same as elsewhere: a finding here is either an additional gap that the upstream chapter did not close, or it is already counted under the chapter. Not both.

#	Description	Level
AC.11.1	**Verify that** AI review and assistant bots treat every piece of PR-supplied content (diff, title, description, comments, file contents, commit messages, linked external URLs) as untrusted input, and apply the AISVS C2.1 prompt-injection defenses: instruction-hierarchy enforcement, content sanitization, and indirect-injection detection.	1
AC.11.2	**Verify that** AI review and assistant bot system prompts and policy configurations are integrity-checked at load time (signed, hash-pinned), and that nothing in the repository, in branch contents, in PR-sourced environment variables, or in any other user-controllable input can modify them.	1
AC.11.3	**Verify that** AI review and assistant bots emit only structured, schema-validated output (JSON with an allow-list of fields and actions). Any free-form output is treated as untrusted and never executed as a command, a query, a shell snippet, or a workflow step.	1
AC.11.4	**Verify that** AI review and assistant bots run in network-isolated, least-privilege sandboxes: a dedicated namespace, default-deny egress with an allow-list to approved APIs only, no mounted repository secrets, and ephemeral credentials only.	2
AC.11.5	**Verify that** any privileged action a bot can take (approving a PR, merging, labeling, dismissing reviews, posting comments outside its sandbox, invoking external tools) goes through a separate, audited authorization path. That path is adjudicated by a policy engine, not by the LLM.	2
AC.11.6	**Verify that** AI review and assistant bots log all prompts (including externally sourced context), tool calls, and outputs to tamper-evident storage. Egress patterns (URLs, IPs, DNS, payload sizes) are continuously monitored for exfiltration indicators, with alerting tuned for webhook, paste-site, and bin-service destinations.	2
AC.11.7	**Verify that** AI review bots run in a zero-privilege, read-only shadow mode for untrusted fork PRs. In shadow mode, inline code-generation commentary is restricted and privileged workflow interaction is forbidden, until a repository maintainer has cleared an initial first-time-contributor verification gate.	2
AC.11.8	**Verify that** AI review and assistant bots are subject to continuous adversarial testing: indirect-prompt-injection corpora are replayed against the bot through simulated PRs, issues, and comments. Detection effectiveness is tracked over time, and a regression blocks the model or prompt update that caused it.	3

Mappings & References:

- **AC.11.1:** AISVS C2.1; OWASP LLM Top 10 (2025) LLM01; OWASP Agentic Top 10 (2026) ASI01, ASI06.
- **AC.11.2:** AISVS C2.1; OWASP LLM Top 10 (2025) LLM01; OWASP Agentic Top 10 (2026) ASI01.
- **AC.11.3:** AISVS C7.1; OWASP LLM Top 10 (2025) LLM05; OWASP Agentic Top 10 (2026) ASI02, ASI05.

- **AC.11.4:** AISVS C9.3; OWASP Agentic Top 10 (2026) ASI02, ASI03, ASI05; NIST SP 800-204D (Workload isolation).
- **AC.11.5:** AISVS C9.5, C5.2.5; OWASP ASVS v5 V4 (Access Control); OWASP Agentic Top 10 (2026) ASI02, ASI03.
- **AC.11.6:** OWASP ASVS v5 V8 (Logging & Error Handling); OWASP LLM Top 10 (2025) LLM02; ISO/IEC 27001:2022 A.8.15, A.8.16.
- **AC.11.7:** GitHub Security Lab "Preventing pwn requests" series (Parts 1-4); OWASP Agentic Top 10 (2026) ASI01, ASI03, ASI09; OWASP CI/CD Top 10 CICD-SEC-01.
- **AC.11.8:** MITRE ATLAS (Indirect prompt injection); AISVS C2.1, C11.1; OWASP SAMM Security Testing (ST).

AC.12 CI/CD Pipeline Hardening Specific to AI Augmentation

Two kinds of CI/CD pipeline control are in scope for this family: those that AI augmentation *newly requires*, and those that AI augmentation *breaks*. Generic CI/CD hygiene is not in scope here; it is covered elsewhere. Short-lived credentials, immutable action pinning, branch protection, SLSA Build Track L3 provenance, and multi-party production approval are all addressed by OWASP ASVS v5 V10, the OWASP Top 10 CI/CD Security Risks (CICD-SEC-01 through CICD-SEC-10), NIST SP 800-204D, and SLSA v1.2. Adopters implement those baselines and verify them against the originating standards. We do not repeat that assessment here.

#	Description	Level
AC.12.1	**Verify that** workflows triggered by untrusted contributions (GitHub Actions pull_request_target, workflow_run, and equivalent fork-aware triggers in other CI systems) never check out, build, test, or otherwise execute untrusted code in a context that has repository write permissions or access to repository, organization, package-registry, cloud, or deployment secrets. Where a privileged follow-up step is needed, the untrusted contribution is first processed in an unprivileged pull_request workflow, and only validated passive artifacts are passed forward to a separate privileged workflow.	1
AC.12.2	**Verify that** secrets, credentials, and pipeline job tokens are not persisted into workspaces that process AI-touched or fork-originated untrusted code. For example, set persist-credentials: false on checkout where the platform supports it, and scrub CI runners of cached credentials before AI tooling runs.	1
AC.12.3	**Verify that** secrets are not exposed to workflows running code from forks or first-time contributors. Environment-protection rules (or the platform equivalent, such as protected variables and deployment approvals) require a manual approval before any secret-bearing job runs for those contributions. This control pairs with AC.11.7 and AC.13.2. Bot-level enforcement under AC.11.7 does not substitute for the platform-level enforcement required here.	1
AC.12.4	**Verify that** self-hosted or persistent runners used by AI tooling are ephemeral (destroyed after each job), network-segmented, and isolated from production credentials. Persistent or long-lived runners do not process fork PRs or AI-generated untrusted artifacts under any circumstances.	2
AC.12.5	**Verify that** changes to workflow definition files (.github/workflows/*, .gitlab-ci.yml, Jenkinsfile, Argo, Tekton, and equivalents) are detected on every PR and route through an elevated review path that includes a security reviewer, regardless of who the contributor is or whether AI was involved. AI agents must not be granted bypass authority over this review path.	2
AC.12.6	**Verify that** pipeline audit logs (workflow runs, secret access, runner registration, permission grants, OIDC token issuance) are streamed in real time to centralized security monitoring. Detection rules are tuned for AI-augmented threat patterns: bulk PR creation from new accounts, workflow-file modifications in fork PRs, unexpected secret access from AI-runner pools, and unusual egress (webhooks, paste sites, bin services) from AI workloads.	2
AC.12.7	**Verify that** artifacts produced by untrusted PR workflows are treated as untrusted passive data when a privileged follow-up workflow consumes them. The privileged workflow never executes binaries, scripts, packages, caches, or generated workflow fragments that originated in an untrusted contribution.	2
AC.12.8	**Verify that** the remediation of a vulnerable workflow includes invalidating or re-validating any PR that was opened before the fix landed. Without this step, a later commit to the same PR can pick up the stale workflow definition and route around the fix.	2

Mappings & References:

- **AC.12.1:** OWASP CI/CD Top 10 CICD-SEC-01, CICD-SEC-04; GitHub Security Lab "Preventing pwn requests" series; NIST SP 800-204D (Pipeline isolation).
- **AC.12.2:** OWASP CI/CD Top 10 CICD-SEC-02, CICD-SEC-06; GitHub Docs (Automatic token authentication and permissions); NIST SP 800-53r5 AC-6 (Least Privilege).
- **AC.12.3:** OWASP CI/CD Top 10 CICD-SEC-01; GitHub Docs (Approving workflow runs from public forks; Protected environments); GitLab Docs (Protected variables).

- **AC.12.4:** OWASP CI/CD Top 10 CICD-SEC-06; NIST SP 800-204D (Runner isolation); ISO/IEC 27001:2022 A.8.22 (Segregation of Networks).
- **AC.12.5:** OWASP CI/CD Top 10 CICD-SEC-01; NIST SSDF PW.7; ISO/IEC 27001:2022 A.8.32.
- **AC.12.6:** OWASP ASVS v5 V8 (Logging); OWASP CI/CD Top 10 CICD-SEC-10 (Insufficient Logging and Visibility); ISO/IEC 27001:2022 A.8.16.
- **AC.12.7:** GitHub Security Lab "Preventing pwn requests" series; OWASP CI/CD Top 10 CICD-SEC-01; NIST SP 800-204D (Cross-workflow trust boundaries).
- **AC.12.8:** GitHub Security Lab "Preventing pwn requests" Part 4 (Alvaro Munoz, 2025); OWASP CI/CD Top 10 CICD-SEC-01; NIST SSDF RV.1.

AC.13 Adversarial AI Detection in Inbound Contributions

The previous families were about defending your own AI from misuse. This one flips the lens. Here the AI is on the attacker's side, and you are trying to spot the signal in inbound contributions and content. The scenario worth defending against is the one where an attacker uses AI to run fork-and-PR campaigns at scale, with malicious payloads tailored to the target repository.

#	Description	Level
AC.13.1	**Verify that** contribution-velocity and contributor-reputation analytics flag anomalies: bulk PR creation from newly created accounts, coordinated fork waves immediately preceding PRs, PR volumes that are inconsistent with human authorship, and reuse of payload patterns across unrelated repositories.	1
AC.13.2	**Verify that** PRs from first-time or low-reputation contributors require maintainer approval before any privileged workflow processes them. Privileged workflows here include AI review bots, secret-bearing jobs, and external-integration calls.	1
AC.13.3	**Verify that** automated PR pipeline gates detect known indicators of LLM-generated or LLM-assisted malicious payload patterns: registry-confusable or typosquatted dependency names, package references that do not resolve to any published version, and dependencies whose creation, first-publication, or maintainer-change timestamps look anomalous relative to the PR.	2
AC.13.4	**Verify that** detection rules are tagged to MITRE ATT&CK (T1195 Supply Chain Compromise and CI/CD-relevant sub-techniques) and to MITRE ATLAS techniques, maintained for the inbound contribution analysis use case, and reviewed against current threat intelligence.	2
AC.13.5	**Verify that** confirmed or high-confidence adversarial contributions trigger automated containment: block the PR, quarantine the fork, suspend the contributor, notify maintainers, and freeze affected workflow files. Triage decisions feed back into detection tuning.	3
AC.13.6	**Verify that** PR analytics include structural AST profiling and stylometric or entropy-based heuristics tuned to identify LLM-generated code patterns. Detection in this category is still maturing, so compensating controls are accepted in place of high-precision automated detection: mandatory human review on flagged PRs, sandboxed execution of suspect payloads, and deferred merge until additional signals accrue.	3

Mappings & References:

- **AC.13.1:** OWASP CI/CD Top 10 CICD-SEC-01; NIST AI RMF MANAGE; MITRE ATLAS (Reconnaissance).
- **AC.13.2:** GitHub Docs (Approving workflow runs from public forks); OWASP CI/CD Top 10 CICD-SEC-01; NIST SSDF PW.4.
- **AC.13.3:** OWASP LLM Top 10 (2025) LLM03; OWASP CI/CD Top 10 CICD-SEC-03 (Dependency Chain Abuse); NIST SSDF PW.4.
- **AC.13.4:** MITRE ATT&CK T1195; MITRE ATLAS (Technique catalogue); OWASP SAMM Threat Assessment (TA).
- **AC.13.5:** NIST AI RMF MANAGE; ISO/IEC 27001:2022 A.5.25 (Assessment of Information Security Events); OWASP SAMM Incident Management (IM).
- **AC.13.6:** MITRE ATLAS (Adversarial ML output detection, research-edge); OWASP LLM Top 10 (2025) LLM03; NIST SSDF PW.8.

AC.14 Compromise Containment & Automated Remediation

Things go wrong eventually. When an AI-adjacent compromise (a prompt-injected bot, a leaked CI secret, a malicious AI-generated artifact in a build) is suspected or confirmed, the goal is to contain the damage and shorten the recovery.

#	Description	Level
AC.14.1	**Verify that** an incident-response playbook exists for AI-in-pipeline compromise. At minimum it covers: revoking AI-agent credentials, rotating every secret that touched the compromised workflow run, quarantining the compromised artifacts, notifying downstream consumers, notifying regulators where applicable, and preserving prompts, responses, and audit logs for forensics.	1
AC.14.2	**Verify that** any secret that touched a workflow run associated with a suspicious PR, a prompt-injection event, or an AI-agent anomaly is automatically rotated, and that downstream issuers (cloud IAM, package registries, signing-key custodians) are notified of the rotation.	1
AC.14.3	**Verify that** AI agent identities (keys, tokens, OIDC trust grants) can be rapidly revoked and quarantined, with a target time-to-revoke that is written down and tested at least once a year.	2
AC.14.4	**Verify that** build provenance and AI BOM records are used during incident response to identify every downstream artifact produced under the suspect AI agent or the compromised pipeline run, so that recall, rebuild, or quarantine can be targeted.	2
AC.14.5	**Verify that** automated remediation is tested in tabletop or live-fire exercises at least once a year. The scenarios include a prompt-injected reviewer bot, fork-PR secret exfiltration, and an AI-generated malicious workflow file.	3

Mappings & References:

- **AC.14.1:** ISO/IEC 27001:2022 A.5.24, A.5.26; NIST AI RMF MANAGE; OWASP SAMM Incident Management (IM).
- **AC.14.2:** OWASP ASVS v5 V6 (Cryptography), V14; OWASP CI/CD Top 10 CICD-SEC-06; NIST SSDF RV.2.
- **AC.14.3:** AISVS C9.4 (Agent and Orchestrator Identity); NIST SP 800-207 (Zero Trust Architecture); ISO/IEC 27001:2022 A.5.18 (Access Rights).
- **AC.14.4:** OWASP SCVS (Bill-of-materials analysis); CycloneDX ML-BOM tracing; NIST SSDF RV.1.
- **AC.14.5:** NIST SSDF RV.1; ISO/IEC 27001:2022 A.5.28 (Collection of Evidence); OWASP SAMM Incident Management (IM).